SR UNIVERSITY

WARANGAL, TELANGANA



FOREST RUNNER – TREE VILLAGE

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

PULLURU LAHARI – 2303A51071

Under the Guidance of

DR. VIJAYA CHANDRA JADALA

ASSISTANT PROFESSOR,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SR UNIVERSITY

ACADEMIC YEAR 2026–2027

CHAPTER 1

INTRODUCTION TO GAME DESIGN AND DEVELOPMENT

1.1 Introduction to Game Design and Development

Game design and development is the process of creating interactive digital experiences by combining programming, graphics, animation, sound, and user interaction. Modern game development uses specialized game engines to create virtual environments and implement gameplay mechanics.

The project Forest Runner – Tree Village is a 3D adventure runner game developed using the Unity game engine and C# programming language. The game is designed around a forest-themed environment in which the player automatically runs along a pathway, avoids obstacles, collects coins, and progresses towards the Tree Village destination.

The project demonstrates the practical implementation of fundamental game development concepts, including player movement, jumping, collision detection, object collection, scoring, user interface design, audio integration, and level completion.

1.2 Fundamentals of Digital Game Design

Digital game design involves defining the structure, rules, objectives, environment, characters, and interactions that form a complete gaming experience.

The major fundamentals include:

- Game Mechanics: Define player actions and interactions, such as running, jumping, collecting coins, and avoiding obstacles.
- Game Environment: Represents the virtual world where gameplay takes place.
- Character Design: Defines the appearance and movement of the player character.
- User Interface: Provides information about the game, including score, collected coins, and game status.
- Audio and Visual Effects: Improve the gaming experience through background music, sound effects, lighting, and visual feedback.
- Game Objectives: Establish the tasks and goals that players must achieve.

1.3 Purpose of the Project

The purpose of Forest Runner – Tree Village is to design and develop an interactive 3D game that demonstrates the application of game development principles using Unity and C#.

The game provides a forest-themed adventure in which players navigate through a pathway, avoid obstacles, collect coins, and reach the final Tree Village destination.

1.4 Objectives of the Project

The primary objectives are:

1. To develop a playable 3D game using Unity.
2. To implement automatic player running.
3. To provide left and right lane-switching functionality.
4. To implement jumping mechanics.
5. To create an engaging forest and tree village environment.
6. To implement obstacles and collision detection.
7. To develop coin collection and score calculation systems.
8. To implement game-over and restart functionality.
9. To design interactive main menu and level-completion screens.
10. To integrate audio and visual elements.
11. To test and deploy the game as a Windows application.

1.5 Significance of the Project

This project provides practical knowledge of 3D game development and demonstrates the integration of different software components into a playable application.

It helps in understanding programming logic, object-oriented development, physics-based interactions, real-time user input, interface design, and game testing.

1.6 Scope of the Project

The project focuses on developing a single playable 3D forest adventure level.

The scope includes:

- Forest-themed environment creation.
- Automatic player movement.
- Lane switching and jumping.
- Obstacle avoidance.

- Coin collection and score tracking.
- Collision detection.
- Game-over and restart screens.
- Level completion functionality.
- Main menu and gameplay interface.
- Audio integration.
- Windows desktop deployment.

Features such as multiplayer support, online leaderboards, and multiple independent levels are outside the current implementation scope and can be considered for future enhancement.

1.7 Expected Outcomes

The expected outcome is a functional 3D runner game that allows players to start the game, control movement, avoid obstacles, collect coins, achieve scores, and complete the level by reaching the Tree Village destination.

The project is expected to demonstrate the successful integration of gameplay mechanics, environment design, user interface elements, and C# scripting within Unity.

1.8 Applications of Game Design and Development

The concepts implemented in this project can be applied in:

- 3D entertainment and adventure games.
- Educational and learning-based games.
- Interactive simulations.
- Virtual environment development.
- Real-time interactive applications.

1.9 Chapter Summary

This chapter introduced the fundamentals of digital game design and development. It explained the purpose, objectives, significance, scope, and expected outcomes of Forest Runner – Tree Village. The following chapters discuss the literature review, project planning, game mechanics, environment design, architecture, implementation, testing, and evaluation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING GAME SYSTEMS

2.1 Introduction

A literature review helps in understanding existing game development techniques, gameplay systems, design approaches, and technologies used in similar applications. Studying existing games provides useful insights into player interaction, environment design, obstacle management, scoring systems, and user experience.

Forest Runner – Tree Village is developed as a 3D runner-style adventure game. This chapter reviews the concepts and characteristics of existing runner games and examines the technologies and techniques relevant to the proposed project.

2.2 Review of Existing Runner Games

Several popular games provide examples of runner-style gameplay. They demonstrate different approaches to player movement, obstacle avoidance, coin collection, environment design, and progression.

2.2.1 Subway Surfers

Subway Surfers is a popular endless-runner game in which players control a character moving through railway tracks. The gameplay involves switching between lanes, jumping over obstacles, sliding under barriers, and collecting coins.



Relevant concepts:

- Lane-based movement.
- Obstacle avoidance.
- Coin collection.
- Score tracking.
- Increasing gameplay difficulty.

These concepts are relevant to the movement and collection mechanics implemented in Forest Runner – Tree Village.

2.2.2 Temple Run

Temple Run is an adventure-based endless-runner game in which players navigate through an environment while avoiding obstacles and collecting rewards.



Relevant concepts:

- Continuous running.
- Jumping and obstacle avoidance.
- Environment-based gameplay.
- Distance and score progression.
- Game-over conditions.

The game provides an example of how an adventure environment can be combined with continuous movement to create an engaging experience.

2.2.3 Minion Rush

Minion Rush is a 3D runner game that combines character movement, obstacle avoidance, collectibles, and colourful environments.



Relevant concepts:

- Three-dimensional environment design.
- Character animation.
- Collectible objects.
- Visual effects.
- User-friendly gameplay interface.

These concepts provide useful references for designing an attractive and interactive 3D game environment.

2.3 Review of Game Development Technologies

2.3.1 Unity Game Engine

Unity is a widely used game development engine that supports the creation of 2D and 3D interactive applications. It provides tools for scene management, GameObjects, components, physics, lighting, animation, and user interface development.

Unity was selected for this project because it supports rapid development and allows different gameplay systems to be integrated within a single environment.

2.3.2 C# Programming Language

C# is used within Unity to implement game logic and control the behaviour of game objects.

In this project, C# scripts are used for player movement, camera following, obstacle behaviour, coin collection, scoring, game management, and user interface interactions.

2.3.3 Unity GameObjects and Components

Unity uses a GameObject-based architecture. GameObjects represent entities in a scene, while components define their properties and behaviour.

This approach helps organize the player, environment objects, obstacles, collectibles, camera, and interface elements.

2.4 Existing Game Systems and Techniques

The following techniques are commonly used in runner-style games:

1. **Player Movement:** Allows the player to move continuously through the game environment.
2. **Lane Switching:** Enables movement between predefined paths.

3. **Jump Mechanics:** Allows players to cross obstacles or gaps.
4. **Collision Detection:** Identifies interactions between the player and other objects.
5. **Collectible System:** Rewards players for collecting objects such as coins.
6. **Scoring System:** Calculates player performance using collected objects and gameplay progress.
7. **Game State Management:** Controls transitions between menu, gameplay, game-over, and completion states.
8. **Camera Following:** Maintains a suitable view of the player during gameplay.

2.5 Limitations of Existing Game Approaches

Some common challenges in runner-style game development include:

- Maintaining responsive player controls.
- Balancing obstacle placement and difficulty.
- Preventing unfair collisions.
- Maintaining smooth gameplay performance.
- Creating visually engaging environments without excessive resource usage.
- Providing clear feedback through the user interface.

These challenges are important considerations when developing a playable and user-friendly game.

2.6 Proposed System Compared with Existing Games

Forest Runner – Tree Village adopts selected concepts from existing runner games while presenting them in a forest-themed environment.

Feature	Existing Runner Games	Forest Runner – Tree Village
Gameplay style	Runner-based gameplay	3D runner adventure
Player movement	Continuous movement	Automatic running
Lane movement	Available in some games	Left and right lane switching

Jumping	Obstacle avoidance	Jump-based obstacle avoidance
Collectibles	Coins and rewards	Coin collection
Environment	Railway, temple, or other themed environments	Forest and Tree Village theme
Scoring	Score and reward systems	Score and coin tracking
Completion	Varies by game	Reach the Tree Village destination

2.7 Advantages of the Proposed System

The proposed game provides the following advantages:

- A simple and understandable control system.
- A visually distinct forest-themed environment.
- Integration of running, jumping, and lane-switching mechanics.
- Coin collection and score tracking.
- Interactive game-over and level-completion screens.
- A playable Windows desktop build.
- A modular structure using Unity C# scripts.

2.8 Chapter Summary

This chapter reviewed existing runner-style games, including Subway Surfers, Temple Run, and Minion Rush. It also discussed Unity, C#, GameObjects, components, and commonly used gameplay techniques. The comparison helped identify relevant design concepts for Forest Runner – Tree Village and provided a foundation for the planning and design decisions discussed in the next chapter.

CHAPTER 3

GAME CONCEPT AND PROJECT PLANNING

3.1 Introduction

Project planning is an important stage in game development. It defines the game concept, objectives, gameplay style, technical requirements, tools, and development activities. Proper planning helps organize the development process and ensures that the final application meets its intended objectives.

Forest Runner – Tree Village is a 3D runner-style adventure game developed using Unity and C#. The game focuses on continuous player movement, obstacle avoidance, coin collection, and reaching a final destination within a forest-themed environment.

3.2 Game Concept

The game is designed around a forest adventure concept. The player character automatically runs through a forest pathway while encountering obstacles and collectible coins.

The primary objective is to avoid obstacles, collect as many coins as possible, achieve a high score, and reach the Tree Village finish area.

3.3 Game Genre

The project belongs to the following game categories:

- 3D adventure game.
- Endless-runner-inspired gameplay.
- Single-player desktop game.
- Casual entertainment game.

3.4 Game Storyline

The player begins an adventure through a forest pathway with the goal of reaching the Tree Village. During the journey, the player must navigate through different obstacles, collect coins, and avoid collisions.

The journey ends when the player reaches the designated finish area or encounters a game-over condition.

3.5 Target Audience

The game is intended for casual players and individuals interested in simple 3D adventure and runner-style games. Its straightforward controls make the gameplay accessible to beginners.

3.6 Technical Requirements

Hardware requirements:

- Windows-compatible computer.
- Keyboard for player input.
- Sufficient RAM and storage for Unity and the game build.
- Graphics hardware capable of running a basic 3D Unity application.

Software requirements:

- Unity 6 game engine.
- C# programming language.
- Visual Studio or another compatible code editor.
- Windows operating system.
- Git and GitHub for source-code management.

3.7 Game Design Document

The major elements of the game's design are:

Element	Description
Game title	Forest Runner – Tree Village
Genre	3D runner adventure
Platform	Windows PC
Game engine	Unity

Programming language	C#
Main objective	Reach Tree Village
Player actions	Run, switch lanes, jump
Collectibles	Coins
Challenges	Obstacles and gaps
Completion	Reach the finish area

3.8 Project Development Plan

The project was organized into the following development stages:

1. Game concept identification.
2. Project creation and Unity configuration.
3. Environment and level construction.
4. Player movement implementation.
5. Obstacle and coin system development.
6. Camera and user interface integration.
7. Audio integration.
8. Gameplay testing and debugging.
9. Windows build generation.
10. Documentation and source-code publication.

3.9 Development Tools

Unity was used for scene creation, GameObject management, scripting, lighting, and application building. C# was used for implementing gameplay logic. GitHub was used to maintain and publish the project source code.

3.10 Chapter Summary

This chapter described the game concept, genre, storyline, target audience, technical requirements, design document, and project planning process. It establishes the development plan followed for Forest Runner – Tree Village.

CHAPTER 4

GAME MECHANICS DESIGN

4.1 Introduction

Game mechanics define the rules and interactions that control how a player experiences a game. They determine the available actions, challenges, rewards, and conditions for success or failure.

Forest Runner – Tree Village uses runner-style mechanics that allow the player to move through a forest environment while avoiding obstacles and collecting coins.

4.2 Player Movement

The player character automatically moves forward during gameplay. This automatic movement creates a continuous runner-style experience.

The player can also switch between available lanes to avoid obstacles and collect coins.

4.3 Lane Switching

Lane switching allows the player to move left or right using keyboard input.

The implemented controls are:

- A or Left Arrow: Move left.
- D or Right Arrow: Move right.

This mechanic helps the player navigate around obstacles placed along the pathway.

4.4 Jumping Mechanics

Jumping allows the player to move vertically to cross obstacles and gaps.

The controls include:

- Spacebar.
- W key.
- Up Arrow key.

Jumping introduces an additional level of interaction and requires the player to respond to upcoming obstacles.

4.5 Obstacle Mechanics

Obstacles are placed throughout the forest environment to challenge the player. The player must avoid colliding with these objects.

Collisions with obstacles or falling can result in a game-over condition.

4.6 Coin Collection System

Coins are collectible objects placed along the player's path. When the player collects a coin, the coin count and score are updated.

The collection system provides an additional objective apart from reaching the destination.

4.7 Scoring System

The game uses a scoring mechanism based on gameplay progress and collected coins.

The scoring elements include:

- Points for collected coins.
- Points based on distance travelled.
- A completion bonus for reaching the finish area.

The implemented game awards 50 points per coin, 1 point per metre, and a 1000-point finish bonus.

4.8 Difficulty Progression

The game's forward movement speed increases during gameplay. This creates a gradually more challenging experience and requires the player to react quickly.

4.9 Win and Lose Conditions

Win condition: The player reaches the Tree Village finish area and triggers the level-completion state.

Lose condition: The player collides with an obstacle or falls, resulting in the game-over state.

4.10 Chapter Summary

This chapter explained the major gameplay mechanics, including automatic running, lane switching, jumping, obstacle avoidance, coin collection, scoring, difficulty progression, and win-or-lose conditions.

CHAPTER 5

GAME ENVIRONMENT AND LEVEL DESIGN

5.1 Introduction

Game environment design involves creating the virtual world in which gameplay takes place. It includes the arrangement of terrain, pathways, obstacles, background elements, and destination structures.

Forest Runner – Tree Village uses a forest-themed 3D environment to provide an adventure-based visual experience.

5.2 Environment Design

The environment is designed around a forest pathway surrounded by natural elements. Trees, ground surfaces, decorative objects, and other environmental elements help establish the forest theme.

The environment provides the visual setting for the player's journey.

5.3 Level Structure

The game contains a single playable level that guides the player from the starting area towards the Tree Village destination.

The level includes:

- Starting area.
- Forest pathway.
- Obstacles.
- Coin placement areas.

- Finish area.

5.4 Terrain and Pathway Design

The pathway acts as the main movement area for the player. It provides a route through the forest and supports lane-based movement.

The pathway layout is designed to make navigation understandable while presenting obstacles and collectibles.

5.5 Obstacle Placement

Obstacles are distributed along the player's route. Their placement creates situations in which the player must switch lanes or jump.

Examples of obstacles include rocks, logs, and gaps.

5.6 Collectible Placement

Coins are positioned throughout the level to encourage players to explore the available movement paths and improve their scores.

5.7 Destination Design

The Tree Village finish area acts as the final objective of the level. Reaching this area triggers the level-completion screen.

5.8 Environment Optimization

The environment uses relatively simple 3D objects and materials to maintain a manageable level of graphical complexity.

This approach supports the objective of creating an attractive game that can run on ordinary desktop hardware.

5.9 Chapter Summary

This chapter described the forest environment, level structure, pathway, obstacle placement, collectible placement, destination, and environment design considerations.

CHAPTER 6

GAME ASSETS AND CHARACTER DESIGN

6.1 Introduction

Game assets are the visual and audio elements used to construct a game. They include characters, models, textures, materials, animations, sounds, and interface graphics.

The project uses Unity objects, materials, procedural construction, and audio files to create its gameplay environment.

6.2 Game Assets

The major asset categories used in the project include:

- Player character and related visual elements.
- Forest environment objects.
- Pathway and ground elements.
- Obstacles such as rocks and logs.
- Coin collectibles.
- Tree Village finish structure.
- User interface graphics.
- Audio resources.

6.3 Character Design

The player character represents the main controllable entity in the game. Its behaviour is controlled using C# scripts.

The character supports automatic running, lane switching, and jumping.

6.4 Materials and Textures

Materials determine the visual appearance of objects in the Unity scene. They control properties such as surface colour and appearance.

Materials are applied to environmental objects and other game elements to maintain a consistent forest-themed visual style.

6.5 Asset Creation and Integration

Unity GameObjects and procedural mesh construction are used to create and organize different visual elements.

The project includes scripts such as MeshFactory and LevelBuilder to support the construction of game objects and the environment.

6.6 Animation

Animation and movement-related behaviour help make the game visually interactive.

The project includes a PlayerAnimator script for player animation-related behaviour and a Bobber script for object movement effects.

6.7 Lighting

Lighting helps establish the appearance and atmosphere of the forest environment. Appropriate lighting improves object visibility and visual presentation.

6.8 Audio Assets

The project includes audio resources for actions and events such as jumping, collecting coins, hitting obstacles, winning, and background music.

6.9 Chapter Summary

This chapter discussed the game assets, character behaviour, materials, procedural construction, animation-related components, lighting, and audio resources used in the project.

CHAPTER 7

GAME ARCHITECTURE AND SYSTEM DESIGN

7.1 Introduction

Game architecture defines how the different systems of a game are organized and communicate with one another.

Forest Runner – Tree Village uses Unity's GameObject and component-based architecture with C# scripts to manage gameplay functionality.

7.2 Overall System Architecture

The game can be represented through the following logical structure:

Player Input → Player Controller → Gameplay Systems → Game State Management → User Interface

The player provides input through the keyboard. The PlayerController processes movement-related behaviour. Gameplay systems manage obstacles, coins, and the camera. The GameManager and UIManager coordinate game states and interface updates.

7.3 Major System Components

7.3.1 PlayerController

The PlayerController script manages player-related movement and control functionality, including lane switching and jumping.

7.3.2 CameraFollow

The CameraFollow script manages the camera's following behaviour so that the player remains visible during gameplay.

7.3.3 GameManager

The GameManager coordinates major game states and gameplay-related management.

7.3.4 UIManager

The UIManager handles gameplay interface updates and presentation.

7.3.5 LevelBuilder

The LevelBuilder script supports construction of the game environment and level elements.

7.3.6 Coin and Obstacle

The Coin and Obstacle scripts manage collectible and obstacle-related behaviours.

7.3.7 FinishLine

The FinishLine script manages the finish-area interaction associated with level completion.

7.3.8 AudioManager

The AudioManager manages the game's audio-related functionality.

7.4 Gameplay Flow

The general gameplay flow is:

1. Launch the application.
2. Display the main menu.
3. Select Start Game.
4. Begin player movement.
5. Switch lanes and jump to avoid obstacles.
6. Collect coins and increase the score.
7. Continue towards the finish area.
8. Display Game Over if the player fails.
9. Display Level Complete if the finish area is reached.
10. Restart or return to the main menu.

7.5 Game State Design

The game uses different states to manage the player experience.

State	Description
Main Menu	Displays the initial game interface
Gameplay	Runs the active game
Game Over	Displays the failure result
Level Complete	Displays successful level completion

7.6 Component Relationships

Unity components communicate through scripts, object references, and event or interaction logic.

For example, the player interacts with obstacle and coin objects during gameplay. These interactions affect the game state and the information displayed by the user interface.

7.7 Chapter Summary

This chapter described the overall architecture, major C# components, gameplay flow, game states, and relationships between the different systems.

CHAPTER 8

GAME DEVELOPMENT USING UNITY AND C#

8.1 Introduction

Unity is a cross-platform game development engine used to create interactive 2D and 3D applications. It provides an editor for scene construction, object management, scripting, physics, animation, lighting, and application deployment.

The Forest Runner – Tree Village project was developed using Unity 6 and C#.

8.2 Unity Project Setup

The project was opened in Unity Hub using the Unity Editor. After the project was loaded, Unity imported the project assets and initialized the game environment.

The project uses the Built-In Render Pipeline.

8.3 Project Structure

The project contains the following major directories:

- **Assets:** Contains scripts, audio, and other game resources.
- **Packages:** Contains package configuration.
- **ProjectSettings:** Contains Unity project configuration.

- **README.md:** Contains project description, controls, and setup instructions.

8.4 GameObjects and Components

Unity uses GameObjects to represent objects within a scene. Components provide the properties and behaviours associated with these objects.

The project uses GameObjects for the player, camera, environment, obstacles, collectibles, and interface elements.

8.5 C# Scripting

C# scripts define the behaviour of game objects and control the interactions between gameplay systems.

The major scripts include:

Script	Purpose
PlayerController.cs	Player movement and control
CameraFollow.cs	Camera following
GameManager.cs	Game management
UIManager.cs	User interface management
LevelBuilder.cs	Environment construction
Coin.cs	Coin behaviour
Obstacle.cs	Obstacle behaviour

FinishLine.cs	Finish-area interaction
AudioManager.cs	Audio management
PlayerAnimator.cs	Player animation behaviour

8.6 Input Handling

Keyboard input is used to control the player.

Key	Action
A / Left Arrow	Move left
D / Right Arrow	Move right
Space / W / Up Arrow	Jump
Enter / Space	Start game
R / Enter	Restart
Esc	Return to main menu

The availability of these controls depends on the current game state.

8.7 Gameplay Implementation

The gameplay implementation combines player movement, obstacle avoidance, coin collection, scoring, camera following, and game state management.

These systems work together to provide the complete runner-style experience.

8.8 Building the Windows Application

After testing the game in the Unity Editor, a Windows desktop build was generated.

The build includes the executable file and the associated Unity data and runtime files required to run the game.

8.9 Chapter Summary

This chapter explained the Unity development environment, project structure, GameObjects, C# scripting, keyboard input, gameplay integration, and Windows build generation.

CHAPTER 9

PHYSICS, ANIMATION, AND COLLISION DETECTION

9.1 Introduction

Physics, animation, and collision detection are important elements of 3D game development. They help create realistic movement, interactive objects, and responses to player actions.

9.2 Player Movement and Physics

The player movement system supports automatic forward movement, lane switching, and jumping.

Jumping introduces vertical movement, allowing the player to cross obstacles and gaps.

9.3 Collision Detection

Collision detection identifies interactions between the player and other objects in the environment.

In the project, collision-related interactions are used for obstacle encounters and coin collection.

9.4 Obstacle Interaction

When the player collides with an obstacle or falls, the game can transition to the Game Over state.

This provides a clear failure condition and encourages the player to improve their movement and timing.

9.5 Coin Interaction

Coins act as collectible objects. When the player interacts with a coin, the collection system updates the coin count and score.

9.6 Animation

Animation-related behaviour improves the visual presentation of the player and other objects.

The project includes PlayerAnimator and Bobber scripts to support animation-related and object movement effects.

9.7 Camera Behaviour

The camera follows the player during gameplay. This maintains the player's visibility and provides a continuous view of the environment.

9.8 Physics and Interaction Testing

Movement, jumping, obstacle interaction, and collectible behaviour should be tested under different gameplay conditions to ensure that interactions work correctly.

9.9 Chapter Summary

This chapter discussed player movement, jumping, collision detection, obstacle interactions, coin collection, animation-related behaviour, and camera following.

CHAPTER 10

USER INTERFACE AND USER EXPERIENCE DESIGN

10.1 Introduction

User Interface (UI) design involves creating the visual elements through which players interact with the game. User Experience (UX) design focuses on making the game understandable, accessible, and enjoyable.

10.2 Main Menu

The main menu provides the initial interface when the game starts. It allows the player to begin the game.

10.3 Gameplay Interface

The gameplay interface presents important information while the player is playing.

This includes score and coin-related information.

10.4 Score Display

The score display communicates the player's progress and performance.

The scoring system considers collected coins, distance travelled, and the finish bonus.

10.5 Game Over Screen

When the player encounters a failure condition, the Game Over interface is displayed.

It provides the option to restart the game or return to the main menu.

10.6 Level Completion Screen

When the player reaches the Tree Village finish area, the Level Complete screen is displayed.

It presents the successful completion result and the player's performance.

10.7 Restart Functionality

The restart option allows players to begin a new attempt without manually reopening the application.

Keyboard shortcuts are also provided for relevant game-state interactions.

10.8 User Experience Considerations

The interface is designed to communicate game status clearly. Simple controls, readable score information, and understandable game-state screens help improve usability.

10.9 Chapter Summary

This chapter described the main menu, gameplay interface, score display, game-over screen, level-completion screen, restart functionality, and user experience considerations.

CHAPTER 11

AUDIO AND VISUAL EFFECTS INTEGRATION

11.1 Introduction

Audio and visual effects contribute to the atmosphere and feedback of a digital game. Background music creates an environment, while sound effects help communicate gameplay events.

11.2 Background Music

The project contains a forest background music audio file named ForestMusic.wav.

Background music helps establish the forest adventure atmosphere during gameplay.

11.3 Sound Effects

The project includes sound effects for different gameplay interactions.

Audio File	Associated Event
Jump.wav	Jumping
Coin.wav	Coin collection
Hit.wav	Obstacle hit
Win.wav	Winning or completion

Click.wav Interface interaction

ForestMusic.wav Background music

11.4 Audio Management

The AudioManager script manages audio-related functionality within the game.

ProceduralAudio is also included in the project to support procedural audio behaviour.

11.5 Visual Effects and Lighting

Visual presentation is improved through environment materials, lighting, object appearance, and animation-related effects.

The LightFlicker script is included for lighting-related behaviour.

11.6 Integration of Audio and Visual Feedback

Audio and visual elements are associated with gameplay events to make interactions easier to recognize and more engaging.

11.7 Chapter Summary

This chapter discussed background music, sound effects, audio management, lighting, and visual feedback integration in Forest Runner – Tree Village.

CHAPTER 12

GAMEPLAY IMPLEMENTATION AND INTEGRATION

12.1 Introduction

Gameplay integration involves combining individual systems into a single functional application. It ensures that player controls, environment objects, collectibles, camera behaviour, scoring, and user interface elements operate together.

12.2 Player and Environment Integration

The player movement system is integrated with the forest environment. The camera follows the player while the player navigates through the pathway.

12.3 Obstacle Integration

Obstacle objects are placed along the gameplay route. Their interactions with the player provide the main challenge of the game.

12.4 Coin and Scoring Integration

Coin collection is connected with the scoring system. When coins are collected, the game updates the player's coin count and score.

Distance-based scoring and the finish bonus contribute to the final score.

12.5 Game State Integration

The GameManager and UIManager coordinate gameplay states and interface presentation.

The game transitions between the main menu, active gameplay, game-over state, and level-completion state.

12.6 Finish Line Integration

The finish area provides the final objective of the level. Reaching it triggers the successful completion state.

12.7 Complete Gameplay Sequence

The complete gameplay sequence is:

1. Launch the Windows application.
2. View the main menu.
3. Start the game.
4. Navigate through the forest.
5. Switch lanes and jump.

6. Avoid obstacles.
7. Collect coins.
8. Increase the score.
9. Reach the Tree Village finish area.
10. View the level-completion result.

12.8 Chapter Summary

This chapter described how player movement, environment design, obstacles, collectibles, scoring, camera behaviour, and interface systems are integrated into a complete playable game.

CHAPTER 13

GAME TESTING, DEBUGGING, AND OPTIMIZATION

13.1 Introduction

Game testing is an essential stage of software development. It verifies whether the game behaves as expected and identifies problems that may affect gameplay, usability, or performance.

13.2 Functional Testing

Functional testing checks whether the main game features work correctly.

The following test cases can be used to evaluate the application:

Test ID	Test Scenario	Expected Result
TC01	Launch the game	Main menu appears
TC02	Select Start Game	Gameplay begins
TC03	Press left movement key	Player switches left

TC04	Press right movement key	Player switches right
TC05	Press jump key	Player jumps
TC06	Collect a coin	Coin count and score update
TC07	Hit an obstacle	Game Over appears
TC08	Reach the finish area	Level Complete appears
TC09	Select Restart	New gameplay attempt begins
TC10	Select Main Menu	Main menu appears

These are the functional scenarios to document. Record actual test outcomes from your own testing before marking every case as passed.

13.3 Debugging

Debugging involves identifying and correcting problems in the project.

Typical areas to inspect include:

- Player movement.
- Keyboard input.
- Collision interactions.
- Coin collection.
- Score updates.
- Game-state transitions.
- User interface buttons.
- Audio playback.

13.4 Gameplay Testing

The game was opened in the Unity Editor and tested through its main gameplay sequence. Testing included starting the game, controlling the player, encountering game-over conditions, restarting, and reaching the level-completion screen.

The Windows build was also launched to verify that the standalone application could run.

13.5 Performance Optimization

The project uses relatively simple environment elements and procedural construction to keep the game manageable.

Potential optimization considerations include:

- Avoiding unnecessary GameObjects.
- Reusing materials where suitable.
- Reducing unnecessary visual complexity.
- Avoiding excessive audio playback.
- Monitoring memory and frame rate.
- Testing on the intended desktop hardware.

13.6 Limitations

Detailed performance measurements such as average FPS, memory consumption, and hardware-specific benchmarks should be recorded separately if required. They should not be estimated or fabricated.

13.7 Chapter Summary

This chapter described the testing approach, functional test scenarios, debugging areas, gameplay testing, and optimization considerations for Forest Runner – Tree Village.

CHAPTER 14

RESULTS AND PERFORMANCE EVALUATION

14.1 Introduction

This chapter presents the results obtained from the developed game and evaluates whether the implemented features satisfy the project objectives.

14.2 Game Implementation Results

The Forest Runner – Tree Village project was successfully opened and executed in Unity. The main gameplay features were demonstrated during testing.

The implemented application includes:

- Main menu.
- Automatic player movement.
- Lane switching.
- Jumping.
- Obstacle interaction.
- Coin collection.
- Score tracking.
- Game-over screen.
- Restart functionality.
- Level-completion screen.
- Windows desktop build.

14.3 Gameplay Results

During gameplay testing, the player was able to navigate through the forest environment, collect coins, avoid obstacles, and reach the finish area.

The level-completion screen displayed the player's final score and coin-related information.

One observed gameplay result showed a score of 6786, 128 out of 173 coins, and a completion time of 34.0 seconds. This should be included as an example gameplay observation, not as a universal benchmark.

14.4 Objective Evaluation

Project Objective	Evaluation
Develop a 3D Unity game	Implemented
Automatic player running	Implemented

Lane switching	Implemented
Jumping	Implemented
Obstacle interaction	Implemented
Coin collection	Implemented
Score tracking	Implemented
Game-over functionality	Demonstrated
Restart functionality	Demonstrated
Level completion	Demonstrated
Windows application build	Generated

14.5 Screenshots of Results

Insert actual screenshots from your game under the following captions:

Figure 14.1:

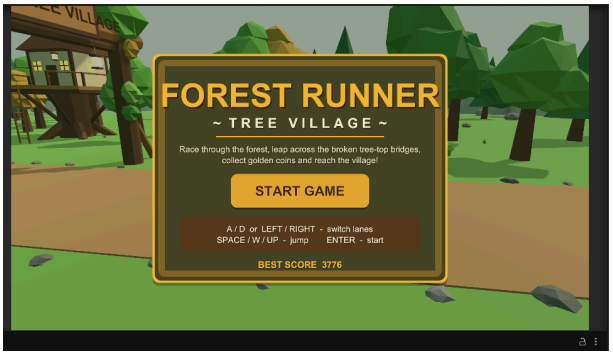


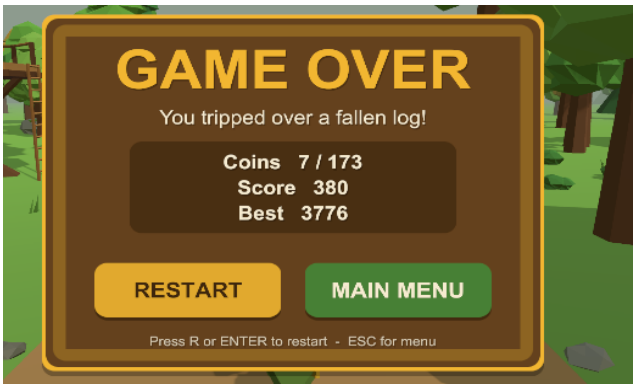
Figure 14.2:



Figure 14.3:



Figure 14.4:



14.6 Performance Evaluation

Performance evaluation should consider:

- Gameplay responsiveness.
- Player control accuracy.
- Stability of game-state transitions.
- Visual quality.
- Loading and execution behaviour.
- Frame rate and memory usage, where measured.

Actual numerical performance results should be added after testing on the target computer.

14.7 Chapter Summary

This chapter presented the implemented game features, gameplay observations, objective evaluation, and screenshots that demonstrate the final application.

CHAPTER 15

PROJECT CONCLUSION AND FUTURE ENHANCEMENTS

15.1 Conclusion

Forest Runner – Tree Village was developed as a 3D runner-style adventure game using Unity and C#. The project demonstrates the practical application of important game development concepts, including player movement, jumping, lane switching, obstacle interaction, coin collection, scoring, user interface design, audio integration, and level completion.

The game provides a forest-themed environment in which players navigate through a pathway, avoid obstacles, collect coins, and reach the Tree Village destination.

The development process provided practical experience in Unity project configuration, C# scripting, GameObject management, gameplay integration, testing, debugging, and Windows application deployment.

The project successfully demonstrates how multiple game development components can be integrated into a single playable desktop application.

15.2 Project Achievements

The major achievements include:

1. Development of a playable 3D runner-style game.
2. Implementation of automatic player movement.
3. Integration of lane switching and jumping.
4. Creation of a forest-themed game environment.
5. Implementation of obstacles and collectible coins.
6. Development of scoring and game-state systems.
7. Creation of game-over and level-completion screens.

8. Integration of background music and sound effects.
9. Generation of a Windows desktop build.
10. Publication of the project source code on GitHub.

15.3 Challenges Faced

The development process involved understanding the Unity Editor, configuring the project, integrating C# scripts, organizing game objects, testing interactions, and generating a standalone build.

Testing was important to verify that the main gameplay sequence and interface transitions worked as expected.

15.4 Limitations

The current project focuses on one playable level and basic runner-style gameplay.

The following features are not part of the current implementation:

- Multiplayer functionality.
- Online leaderboards.
- Multiple independent levels.
- Advanced enemy artificial intelligence.
- Mobile platform deployment.

15.5 Future Enhancements

The project can be extended through the following features:

15.5.1 Multiple Levels

Additional environments, pathways, and difficulty settings can be introduced to provide more gameplay variety.

15.5.2 Advanced Character Animations

More detailed running, jumping, landing, and movement animations can improve character presentation.

15.5.3 Power-Ups

Additional collectible items and temporary abilities can introduce new gameplay strategies.

15.5.4 Multiplayer Support

A multiplayer mode can allow players to compete or cooperate within the game environment.

15.5.5 Online Leaderboards

An online leaderboard can store and display player scores.

15.5.6 Mobile Platform Support

The game can be adapted for Android and iOS devices by introducing touch-based controls and platform-specific optimization.

15.5.7 Enhanced Visual Effects

Additional particle effects, environmental animations, lighting improvements, and weather effects can improve the visual experience.

15.6 Learning Outcomes

The project provided practical knowledge of:

- 3D game development using Unity.
- C# programming.
- GameObject and component-based architecture.
- Player movement and input handling.
- Collision detection and interaction systems.
- User interface development.
- Audio integration.
- Game testing and debugging.
- Windows application deployment.
- Source-code management using GitHub.

15.7 Final Summary

Forest Runner – Tree Village demonstrates the development of a complete playable 3D game using Unity and C#. The project meets its primary objective of implementing an interactive forest adventure with movement, obstacles, collectibles, scoring, and level completion. It also provides a foundation for future improvements and more advanced game development features.